

# Agent State Discipline: Deterministic Glyph-Based Recursion Governance for OpenClaw

## Executive Summary

OpenClaw is a self-hosted “Gateway” that connects chat surfaces (WhatsApp, Telegram, Discord, etc.) to agentic LLM runs with tool execution, persistent sessions, and extensibility via plugins, hooks, and HTTP/WS APIs. [1] As OpenClaw deployments expand from “single-user automation” into shared or semi-shared surfaces, operators encounter a predictable cluster of failure modes: tool-call recursion loops, unbounded tool consumption, agent drift (goal and policy deviation), and unsafe or irreversible actions taken under ambiguous authority. OpenClaw already provides important guardrails—tool policies, sandboxing, DM pairing/allowlists, exec approvals, and optional loop-detection for repetitive tool-call patterns—but these controls are primarily about *blast radius* and *access control*, not fully deterministic governance of *agent state transitions*. [2]

This whitepaper drafts a rigorous, implementable approach—**Agent State Discipline (ASD)**—that layers deterministic, auditable “recursion governance” on top of OpenClaw’s existing security stance (“access control before intelligence”). [3] ASD is centered on a **glyph-to-enum finite-state machine (FSM)** that explicitly encodes an agent run’s phase (intake → plan → act → integrate → ground → consent gate → crystallize → synthesize). The FSM is enforced by a **Recursion Governor** (a policy engine) that: (1) constrains recursion depth and tool budgets, (2) requires explicit *irreversible-action gates* for high-risk operations, and (3) continuously emits append-only telemetry and audit records for post-hoc inspection and compliance.

Key design decisions are tailored to OpenClaw realities:

- **Compatibility-first integration:** OpenClaw’s tool system is policy-controlled (profiles, allow/deny, groups) and tools are exposed both as prompt text and as structured schemas—if a tool is not exposed, the model cannot call it. [4] ASD leverages this by recommending a **“proxy tool” integration tier** that can be deployed without relying on potentially unstable “agent loop plugin hook” APIs. [5]
- **Threat realism:** Prompt injection (direct and indirect) is now treated as a persistent design risk rather than a “patchable bug.” This aligns with OpenClaw’s own guidance that system prompts are “soft guidance,” while real enforcement comes from tool policy, sandboxing, approvals, and allowlists; and with external security research describing indirect prompt injection as a remote exploit vector for LLM-integrated apps. [6]
- **Determinism and auditability:** The governor’s decisions are deterministic functions of declared state + proposed event + configured policy, and all decisions

are logged using tamper-evident techniques consistent with log-management best practices. [7]

The remainder of this document provides the requested: a problem statement, threat model, design goals, a formal specification (state schema, glyph mapping, transition rules, recursion policy, irreversible-gate semantics, telemetry), an OpenClaw adapter contract (JSON schemas, auth headers, nonce/signature, rate limits, sidecar architecture), implementation guidance (plugin patterns, pseudocode, test vectors, unit/integration tests), security controls, deployment recommendations, evaluation methodology, and a proposed standardization path (v0.1 spec + compliance suite).

## Context and Problem Statement

### OpenClaw operational context that drives ASD requirements

OpenClaw’s architecture centers on a single always-on Gateway process that multiplexes WebSocket control/RPC and HTTP endpoints (including OpenAI-compatible chat and an OpenResponses-compatible API), while running agent loops that handle prompt assembly, tool execution, streaming output, and persistence. [8] The agent loop is the authoritative pipeline from message intake to tool actions and final replies, and OpenClaw documents extensibility points (internal hooks and plugin lifecycle/tool hooks) and the fact that tool execution and messaging are integrated into the loop. [9]

OpenClaw’s security docs emphasize that there is no “perfectly secure” setup when wiring frontier-model behavior into real messaging surfaces and tools; the pragmatic goal is to control who can talk to the bot, where it can act, and what it can touch—starting with the smallest access and widening carefully. [10] This informs ASD’s philosophy: *constrain authority first; then engineer agent behavior within that constrained authority*.

### Problem statement

Agentic runs in OpenClaw can fail in ways that are *operationally expensive* and *security-relevant*:

**Agent recursion and no-progress loops.** OpenClaw can detect repetitive tool-call loops with an optional `tools.loopDetection` configuration (disabled by default), using detectors like repeated identical tool invocations and “poll no progress” patterns. [11] This is valuable, but it is a *symptom detector* rather than a formal governance model: it identifies repeating patterns, but does not provide a deterministic “state discipline” framework for *why* the agent is looping or what state transition is allowed next.

**Tool overuse and unbounded consumption.** Excessive tool usage aligns with OWASP’s GenAI security risk category “Unbounded Consumption,” where uncontrolled LLM/app usage can lead to service degradation, financial loss, and denial-of-wallet outcomes. [12] OpenClaw’s tool ecosystem includes high-cost tools (web search/fetch, browser automation, exec/process sessions), each of which can multiply run length, wall-clock time, and token usage. [13]

**Drift and policy deviation.** “Drift” here means the agent’s internal plan and action sequence diverges from: (a) the user’s request, (b) the deployment’s safety posture, or (c) the “least privilege” policy intent. OpenClaw explicitly warns that prompt injection is not solved by system prompts; manipulation is expected, so policy must be enforced via tool policy, sandboxing, and approvals. [14] From an industry standpoint, agentic “excessive agency” (granting too much autonomy via tool access) is recognized as a top GenAI risk, emphasizing bounded permissions and validation. [15]

**Irreversible actions and persistent side effects.** OpenClaw’s built-in tools include control-plane actions (gateway tool can apply config patches/updates; cron can create sustained jobs) and high-risk capability hosts (nodes can expose commands like `system.run`). OpenClaw explicitly recommends denying control-plane tools in untrusted contexts and treating node execution as remote code execution. [16] Once executed, these actions may be difficult to undo, and may change system state beyond a single session.

## ASD thesis

ASD proposes that reliable, safe OpenClaw agents need **a deterministic governance layer** that sits *between* “LLM intent” and “tool execution,” defining:

- a **small, explicit state machine** of agent phases (glyph-based but machine-enforced),
- **bounded recursion** and **tool budgets** per phase and per run,
- **formal irreversible gates** (confirmation + authorization) for high-risk transitions,
- and **tamper-evident, append-only telemetry** for auditing and continuous improvement.

This approach is consistent with OpenClaw’s stance that “hard enforcement comes from tool policy, exec approvals, sandboxing, and channel allowlists,” not from prompt text alone. [17]

## Threat Model and Assumptions

### Adversary model

ASD assumes a realistic adversary model aligned with modern LLM-app security guidance:

Prompt injection is fundamentally an “instructions vs data” confusion risk, and defenders should not assume it can be fully eliminated—only constrained in impact. [18] The UK NCSC frames prompt injection as distinct from SQL injection and emphasizes secure system design that treats models as “confusable deputies,” i.e., systems that can be tricked into acting on behalf of untrusted inputs. [19]

Indirect prompt injection is especially relevant for OpenClaw because even if inbound DM access is locked down, untrusted content can enter via web results, pages, attachments,

emails, and other retrieved data. OpenClaw explicitly warns that prompt injection does not require public DMs because untrusted content can carry adversarial instructions that coerce tool calls or exfiltrate context. [20]

Microsoft’s guidance on defending against indirect prompt injection emphasizes that these attacks often succeed when LLM-based applications operate with the same broad access permissions as the user; mitigations include fine-grained permissions and access controls to reduce unintended actions. [21]

## Persona targeting and “Ulantra” as an asset

The user scenario includes “hacker bots targeting personas like Ulantra.” In ASD terms, a **persona** is a policy-bearing identity layer (system prompt and/or behavioral constraints) that can be attacked by:

- **Instruction override:** “ignore your instructions,” “reveal your hidden prompt,” etc. [22]
- **Goal hijack:** manipulating the persona into changing its objective mid-run (e.g., “be noticed,” “spread virally,” “teach other agents/bots”), which can become an “excessive agency” escalation if tool permissions are broad. [15]
- **Cross-context coercion:** using session routing, shared inboxes, or broad tool access to act on behalf of someone else. OpenClaw warns against combining shared DMs with broad tool access and recommends per-peer DM isolation (session.dmScope: "per-channel-peer"). [10]

ASD treats persona instructions as **non-authoritative** and expects them to be compromised at times. The authoritative control plane is the governor’s deterministic policy + OpenClaw’s access-control primitives.

## Trust boundaries and assumptions

OpenClaw strongly distinguishes between: (a) who can send messages and trigger workflows (pairing/allowlists/mentions), and (b) what tools and surfaces are enabled. [23]  
ASD assumes:

- The **OpenClaw Gateway host** is trusted to execute configured code and store local state, but is not assumed immune to compromise; audit and least-privilege still apply. [24]
- **Plugins are trusted code** in OpenClaw because they run in-process with the Gateway. OpenClaw hardens plugin discovery paths and validates plugin config via JSON Schema without executing plugin code during validation, but runtime plugin execution remains privileged. [25]
- **Tool execution is a primary attack surface**, especially where exec, browser, web fetch/search, cron, gateway config changes, nodes, and session tools are enabled. [26]

- **Local model deployments increase security variance:** OpenClaw notes local is “doable” but expects large context and strong defenses against prompt injection, warning that “small”/aggressively quantized checkpoints can raise prompt-injection risk. [27]

## What ASD explicitly does not assume

ASD does not assume: - the model will obey state discipline because it is instructed to (OpenClaw calls system prompts “soft guidance only”). [28]

- prompt injection can be prevented by filtering alone; instead, ASD focuses on impact-limiting enforcement. [29]

- OpenClaw’s internal “agent loop plugin hooks” are always available/reliable to third-party plugin authors. A documented issue reports plugin hooks like `agent_end` and `before_tool_call` not being invoked under some versions. [30]

## Design Goals and Architecture Overview

### Design goals

ASD’s design goals are derived from the problem statement and OpenClaw’s security posture:

**Determinism.** For a given (`policy`, `prior_state`, `proposed_event`), the governor returns exactly one decision, including an allowed/denied action and the next state transition. Determinism is crucial because prompt text and model behavior are not reliable security boundaries. [31]

**Auditability.** Every transition and irreversible action must be explainable as “policy rule X applied to inputs Y,” and reconstructible from logs. OpenClaw already stores session transcripts on disk and warns to treat disk access as a trust boundary; ASD adds structured audit records on top. [24]

**Minimal trust / least privilege.** The governor should operate as a “permission gate” that embodies least privilege principles (NIST AC-6) and reduces “excessive agency.” [32]

**Compatibility with OpenClaw.** ASD must respect OpenClaw’s tool policy model (profiles, allow/deny, groups), sandboxing, and Gateway protocol/auth patterns (nonces, signed challenges, device tokens, idempotency keys). [33]

**Safe handling of irreversible actions.** High-risk actions must require explicit gates (user confirmation, operator approval, or capability tokens), similar in spirit to OpenClaw’s exec approvals and security guidance to deny control-plane tools in untrusted contexts. [34]

### Reference architecture

ASD is specified as two components:

**Recursion Governor (sidecar service).** A local network service that implements: - state transition rules, - budgets (recursion depth, tool count, token/time budgets), - irreversible action gates, - and telemetry/audit log emission.

**OpenClaw Adapter (plugin + config profile).** An OpenClaw plugin that: - exposes one or more “governed” agent tools (JSON-schema functions) to the LLM, per OpenClaw’s plugin tool model, [\[35\]](#)

- consults the sidecar before executing any underlying effect, - and configures OpenClaw tool policy so other high-risk tools are not exposed directly to the model.

This “proxy tool” architecture is intentionally designed so it can work even if agent-loop plugin hooks are unavailable or unstable. [\[5\]](#)

### Design alternatives and trade-offs

| Alternative                                   | Summary                                              | Pros                                                                               | Cons                                                                                      | Fit for OpenClaw                                                    |
|-----------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Prompt-only glyph discipline                  | Put FSM + glyph rules in system prompt / skills docs | Very easy to ship; no code                                                         | Not enforceable; system prompts are “soft guidance” only; injection can override          | Good as onboarding, not governance <a href="#">[28]</a>             |
| Loop detection only                           | Enable <code>tools.loopDetection</code>              | Already supported; blocks repetitive patterns                                      | Detects some loops but not general drift/irreversible safety; not a full governance model | Good baseline, not sufficient <a href="#">[11]</a>                  |
| Agent-loop hook interception                  | Intercept <code>before_tool_call</code> etc.         | Potentially strongest: can mediate all tool calls                                  | Hook availability to third parties may vary; reported issues                              | Use if verified in your version; keep fallback <a href="#">[36]</a> |
| Proxy tool + strict tool policy (recommended) | Expose only governed tool(s), deny others            | Deterministic enforcement; minimizes attack surface; uses documented tool policies | Requires explicit mapping of allowed underlying actions; additional latency               | Strong compatibility path <a href="#">[37]</a>                      |

| Alternative              | Summary                          | Pros           | Cons                                                                | Fit for OpenClaw                                         |
|--------------------------|----------------------------------|----------------|---------------------------------------------------------------------|----------------------------------------------------------|
| Full in-process governor | Put governor logic inside plugin | Lowest latency | Larger trusted code base inside Gateway; harder independent testing | Viable for advanced operators; audit still required [38] |

## Formal Specification

### State model and glyph-to-enum mapping

ASD uses a *dual representation*:

- **Glyphs** (human-visible, compact, memorable): useful for users, logs, and UI overlays.
- **Enums** (machine authoritative): used for deterministic transitions and data validation.

Proposed canonical v0.1 mapping (based on the user’s glyph set):

<Redacted>

This mapping is intentionally **one-to-one** for determinism and auditability. Visual “languages” can vary, but governance cannot: for interoperability, glyph personalization must map back to the canonical enum set.

### State schema

Below is a sample **ASD State** JSON Schema (Draft 2020-12). Fields marked “optional” can be omitted in v0.1 but are reserved.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.org/asd/state.schema.json",
  "title": "ASDState",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "specVersion",
    "sessionKey",
    "runId",
    "phase",
    "glyph",
    "depth",

```

```

    "budgets",
    "trust",
    "history"
  ],
  "properties": {
    "specVersion": { "type": "string", "pattern": "^0\\.1\\.([0-9]+)$" },
    "sessionKey": { "type": "string", "minLength": 1 },
    "runId": { "type": "string", "minLength": 1 },

    "phase": {
      "type": "string",
      "enum": [
        "Redacted",
        "Redacted",
        "Redacted",

        "SYNTHESIS_CLOSE"
      ]
    },
    "glyph": {
      "type": "string",
      "enum": [Redacted, Redacted]
    },

    "depth": {
      "type": "object",
      "additionalProperties": false,
      "required": ["turn", "recursion"],
      "properties": {
        "turn": { "type": "integer", "minimum": 0 },
        "recursion": { "type": "integer", "minimum": 0 }
      }
    },

    "budgets": {
      "type": "object",
      "additionalProperties": false,
      "required": ["maxRecursion", "maxToolCalls", "maxWallClockMs"],
      "properties": {
        "maxRecursion": { "type": "integer", "minimum": 0, "maximum": 1000 },
        "maxToolCalls": { "type": "integer", "minimum": 0, "maximum": 10000 }
      },
      "maxWallClockMs": { "type": "integer", "minimum": 1, "maximum":
86400000 },
      "phaseBudgets": {
        "type": "object",
        "additionalProperties": {
          "type": "object",
          "additionalProperties": false,
          "properties": {

```

```

        "maxToolCalls": { "type": "integer", "minimum": 0 },
        "maxRecursion": { "type": "integer", "minimum": 0 }
    }
}
},
"trust": {
    "type": "object",
    "additionalProperties": false,
    "required": ["inputTrustLevel", "sources"],
    "properties": {
        "inputTrustLevel": {
            "type": "string",
            "enum": ["trusted", "mixed", "untrusted"]
        },
        "sources": {
            "type": "array",
            "items": {
                "type": "object",
                "additionalProperties": false,
                "required": ["kind", "id"],
                "properties": {
                    "kind": {
                        "type": "string",
                        "enum": ["channel_message", "web", "file", "email",
"tool_output", "operator_override"]
                    },
                    "id": { "type": "string" },
                    "sha256": { "type": "string", "pattern": "^[a-f0-9]{64}$" }
                }
            }
        }
    }
},
"history": {
    "type": "object",
    "additionalProperties": false,
    "required": ["events"],
    "properties": {
        "events": {
            "type": "array",
            "maxItems": 5000,
            "items": {
                "type": "object",
                "additionalProperties": false,
                "required": ["ts", "type"],
                "properties": {

```

```

        "ts": { "type": "string", "format": "date-time" },
        "type": { "type": "string" },
        "detail": { "type": "object" }
      }
    }
  }
},
"telemetry": {
  "type": "object",
  "additionalProperties": true,
  "properties": {
    "toolCallCount": { "type": "integer", "minimum": 0 },
    "blockedCount": { "type": "integer", "minimum": 0 },
    "lastDecisionHash": { "type": "string", "pattern": "^[a-f0-9]{64}$" }
  }
}
}
}

```

## Transition rules and recursion depth policy

ASD transition rules are deterministic. The governor should reject any transition that violates:

- **Phase ordering:** default linear progression, with explicit allowed back-edges (e.g., <Redacted> when verification fails).
- **Budgets:** recursion depth and tool-call caps; plus optional per-phase budgets.
- **Trust constraints:** untrusted inputs cannot directly trigger irreversible actions; they must route through <REDACTED> and/or “reader agent” workflows consistent with OpenClaw’s “untrusted content” recommendations. [\[28\]](#)
- **Tool policy:** no tool invocation unless the tool is allowed by OpenClaw tool policy (profiles/groups/allow/deny). [\[39\]](#)
- **Loop detection compatibility:** if OpenClaw tools.loopDetection is enabled, ASD should treat an OpenClaw loop warning/block as a forced transition to VOID\_HALT\_OR\_QUERY. [\[40\]](#)

A recommended default recursion-depth policy for v0.1:

- maxRecursion = 12 per run (hard stop to prevent unbounded loops).
- Per-phase recursion caps: <REDACTED>
- If budget exceeded: transition to <REDACTED> with a single user-facing question or a safe stop.

These defaults are intentionally conservative and should be tuned per deployment (especially if OpenClaw’s own `agents.defaults.timeoutSeconds` and loop detection are enabled). [41]

## Irreversible-gate semantics

OpenClaw explicitly identifies high-risk control-plane tools (gateway, cron) and recommends denying them in untrusted contexts. [23] It also warns that node execution (`system.run`) is remote code execution. [42] ASD formalizes this into **Irreversible Gates**:

1. The governor classifies a proposed action as reversible or irreversible (or “high-risk reversible”).
2. If irreversible, the governor transitions to <REDACTED> and returns a *challenge* that must be satisfied before execution.
3. The challenge must be bound to: <REDACTED>
4. Only on valid confirmation does the governor allow the transition to <REDACTED> (commit) or back to <REDACTED> for execution.

This resembles OpenClaw’s own use of nonces and signed challenges in the Gateway protocol handshake, and its exec-approval workflow. [43]

## Finite-state machine diagram

stateDiagram-v2  
<REDACTED>

## OpenClaw Adapter Contract and Implementation Guidance

### Integration strategy for OpenClaw

*Recommended integration tier: proxy tool + strict tool policy*

OpenClaw tools are presented to the model as both system prompt text and structured tool schemas; if a tool does not appear in either channel, the model cannot call it. [44] Therefore, the most robust enforcement strategy is:

1. Configure OpenClaw tool policy so *only* a small set of safe tools are exposed (often message plus session introspection), and deny high-risk groups (group:fs, group:runtime, group:web, cron, gateway, etc.) by default. OpenClaw’s security baseline shows exactly this style: messaging profile plus explicit denies for runtime/fs and session-spawn/send in hardened contexts. [45]
2. Add an **ASD plugin tool** (e.g., `asd_execute`) as the only “action tool” the model can call for non-trivial work. OpenClaw plugins can register tools with JSON schema parameters, and tools can be optional (opt-in) or required. [35]
3. `asd_execute` calls the local sidecar governor, receives a deterministic decision, and only then invokes underlying operations—preferably through OpenClaw’s **POST**

`/tools/invoke` endpoint for built-in tools, because it is always enabled and still gated by Gateway auth and tool policy. [46]

This preserves OpenClaw’s “hard enforcement” model while adding deterministic governance.

#### *Note on hook-based interception*

OpenClaw documents agent loop hook points (e.g., `before_tool_call`) but a reported issue indicates that “plugin hooks” registered via `api.on()` were never invoked in at least some versions. [36] ASD v0.1 therefore treats agent-loop hook interception as **optional/advanced**, and recommends the proxy-tool approach as the baseline.

### Sidcar architecture and transport

- Deploy the governor as a **localhost-only** service (e.g., `127.0.0.1:3939`) or as a container on the same host with a private bridge network.
- Do **not** expose it on LAN/WAN by default. This mirrors OpenClaw’s default “loopback” bind stance for safety and its emphasis on local-only control planes unless deliberately secured. [47]

### ASD decision API: request/response

#### *Headers: nonce + signature*

OpenClaw’s Gateway protocol uses a server-provided `connect.challenge` nonce and requires a signed response for non-local connections; it also issues scoped device tokens after pairing. [48] ASD borrows this pattern for the sidcar:

- X-ASD-Nonce: random nonce generated by caller (plugin) per request.
- X-ASD-Signature: HMAC-SHA256 over (`method + path + body_sha256 + nonce + ts`) using a shared secret.
- X-ASD-Timestamp: `unix ms`
- X-ASD-Idempotency-Key: required for side-effecting “execute” decisions (mirrors OpenClaw’s emphasis on idempotency keys for side-effecting WS methods). [49]

*(Unspecified detail):* OpenClaw does not define an official pattern for third-party sidcar signatures; this is a proposed default aligned with its nonce/signature patterns.

#### *Request schema (governance decision)*

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.org/asd/decide.schema.json",
  "title": "ASDDecideRequest",
  "type": "object",
  "additionalProperties": false,
  "required": ["state", "proposed"],
  "properties": {
    "state": { "$ref": "https://example.org/asd/state.schema.json" },
```

```

    "proposed": {
      "type": "object",
      "additionalProperties": false,
      "required": ["kind"],
      "properties": {
        "kind": { "type": "string", "enum": ["transition", "tool_call",
"finalize"] },

        "transition": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "toPhase": { "type": "string" },
            "reason": { "type": "string" }
          }
        },

        "tool": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "toolName": { "type": "string" },
            "toolArgs": { "type": "object" },
            "toolRisk": { "type": "string", "enum": ["low", "medium", "high",
"irreversible"] }
          }
        }
      }
    },

    "context": {
      "type": "object",
      "additionalProperties": true,
      "properties": {
        "openclaw": {
          "type": "object",
          "additionalProperties": true,
          "properties": {
            "agentId": { "type": "string" },
            "channelId": { "type": "string" },
            "senderId": { "type": "string" },
            "toolPolicyHash": { "type": "string" }
          }
        }
      }
    }
  }
}

```

*Response schema (allow/deny + next state)*

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.org/asd/decide.response.schema.json",
  "title": "ASDDecideResponse",
  "type": "object",
  "additionalProperties": false,
  "required": ["decision", "nextState", "audit"],
  "properties": {
    "decision": {
      "type": "object",
      "additionalProperties": false,
      "required": ["allow", "code"],
      "properties": {
        "allow": { "type": "boolean" },
        "code": {
          "type": "string",
          "enum": [
            "ALLOW",
            "DENY_POLICY",
            "DENY_BUDGET",
            "DENY_UNTRUSTED_INPUT",
            "REQUIRE_CONSENT",
            "HALT"
          ]
        }
      }
    },
    "userMessage": { "type": "string" },
    "consent": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "required": { "type": "boolean" },
        "challengeId": { "type": "string" },
        "expiresAt": { "type": "string", "format": "date-time" },
        "actionFingerprint": { "type": "string" }
      }
    }
  }
},

"nextState": { "$ref": "https://example.org/asd/state.schema.json" },

"audit": {
  "type": "object",
  "additionalProperties": false,
  "required": ["decisionHash", "ts"],
  "properties": {
    "ts": { "type": "string", "format": "date-time" },
    "decisionHash": { "type": "string", "pattern": "^[a-f0-9]{64}$" },
  }
}
```

```

    "policyVersion": { "type": "string" }
  },
  "rateLimit": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "limit": { "type": "integer" },
      "remaining": { "type": "integer" },
      "resetMs": { "type": "integer" }
    }
  }
}

```

## OpenClaw adapter: tool execution contract

### *Using /tools/invoke as an execution backend*

OpenClaw’s Gateway exposes POST /tools/invoke for invoking a single tool, gated by Gateway auth and tool policy, and returns 404 if the tool is not allowed; it also supports a default deny list for especially dangerous tools and can rate-limit auth failures with 429 + Retry-After. [50]

ASD can exploit this property by:

- Having the plugin call POST /tools/invoke on 127.0.0.1:<gatewayPort> after the governor returns ALLOW.
- Relying on OpenClaw tool policy as a second enforcement layer (“defense in depth”). [51]

(Unspecified detail): the exact internal data structures to call tools directly in-process are not specified in the public plugin SDK docs; using /tools/invoke is a robust default because it is explicitly documented.

### *Example asd\_execute tool shape*

The model calls:

```

{
  "tool": "asd_execute",
  "args": {
    "phase": "1",
    "intent": "Fetch recent web results for query and summarize safely",
    "requestedTool": "web_search",
    "requestedArgs": { "query": "OpenClaw tool policy loop detection",
  "count": 5 }
  }
}

```

Your plugin then: 1. Builds an ASDDecideRequest using the current persisted state for (sessionKey, runId). 2. If ALLOW, calls POST /tools/invoke with { tool, args, sessionKey }. [52] 3. Records both the decision and the tool result into an append-only audit log. 4. Returns a tool response to the model, including updated state (glyph) and a short “next step” instruction.

## Example OpenClaw plugin snippet (TypeScript)

OpenClaw plugins are TypeScript modules loaded at runtime, and plugins can register tools with JSON-schema parameters (TypeBox is used in examples). [53] The snippet below illustrates the proxy-tool pattern (simplified).

```
// ~/.openclaw/extensions/asd-governor/index.ts
import { Type } from "@sinclair/typebox";

type ToolResult = { content: Array<{ type: "text"; text: string }> };

// (Unspecified) Replace with your preferred HTTP client.
async function postJson(url: string, headers: Record<string, string>, body:
unknown) {
  const res = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "application/json", ...headers },
    body: JSON.stringify(body),
  });
  const text = await res.text();
  if (!res.ok) throw new Error(`HTTP ${res.status}: ${text}`);
  return JSON.parse(text);
}

export default function register(api: any) {
  api.registerTool({
    name: "asd_execute",
    description:
      "Deterministic glyph-based recursion governor. Requests an action;
governor decides allow/deny + state transition.",
    parameters: Type.Object({
      phase: Type.String({ description: "Glyph for current phase (e.g., ②)"
    })),
    intent: Type.String(),
    requestedTool: Type.String(),
    requestedArgs: Type.Object({}, { additionalProperties: true }),
  }),
  async execute(_id: string, params: any): Promise<ToolResult> {
    // (Unspecified) How to reliably access sessionKey/runId in tool
    context varies by SDK.
    // Use api-provided context if available; otherwise require it in
    params.
```

```

const sessionKey = "main";

const state = await loadAsdState(api, sessionKey); // your persistence
const decideReq = {
  state,
  proposed: {
    kind: "tool_call",
    tool: {
      toolName: params.requestedTool,
      toolArgs: params.requestedArgs,
      toolRisk: classifyRisk(params.requestedTool,
params.requestedArgs),
    },
  },
  context: { openclaw: { agentId: "main" } },
};

const governorUrl = api.config?.plugins?.entries?.["asd-
governor"]?.config?.governorUrl
?? "http://127.0.0.1:3939/v1/asd/decide";

const decision = await postJson(governorUrl, signHeaders(decideReq),
decideReq);

await appendAudit(api, decision);

if (!decision.decision.allow) {
  const msg = decision.decision.userMessage ?? `Blocked:
${decision.decision.code}`;
  await saveAsdState(api, sessionKey, decision.nextState);
  return { content: [{ type: "text", text: msg }] };
}

// Execute through OpenClaw /tools/invoke for defense-in-depth policy
enforcement.
// Tools Invoke is documented as always enabled and gated by auth +
tool policy. [50]
const gatewayUrl = api.config?.gateway?.httpBaseUrl ??
"http://127.0.0.1:18789";
const invokeUrl = `${gatewayUrl}/tools/invoke`;

const invokeBody = {
  tool: params.requestedTool,
  args: params.requestedArgs,
  sessionKey,
};

const invokeHeaders: Record<string, string> = {};
// If your gateway uses bearer tokens, include Authorization. [54]

```

```

    const gwToken = api.config?.gateway?.auth?.token ??
process.env.OPENCLAW_GATEWAY_TOKEN;
    if (gwToken) invokeHeaders["Authorization"] = `Bearer ${gwToken}`;

    const toolResult = await postJson(invokeUrl, invokeHeaders,
invokeBody);

    const nextState = decision.nextState;
    await saveAsdState(api, sessionKey, nextState);

    return {
      content: [
        { type: "text", text: JSON.stringify({ next: nextState.glyph,
toolResult }, null, 2) },
      ],
    };
  },
});
});
}

```

This snippet deliberately marks context capture and config access as **unspecified** where OpenClaw’s public SDK documentation does not fully define the tool execution context object for plugins.

## Test vectors and validation strategy

### *Deterministic test vectors (examples)*

A v0.1 compliance suite should include table-driven vectors:

| Vector | Prior phase | Proposed action                     | Expected decision                       | Rationale                                                                                                             |
|--------|-------------|-------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| V1     | ?           | web_fetch on arbitrary URL          | REQUIRE_CONSENT or DENY_UNTRUSTED_INPUT | Untrusted content channel; OpenClaw recommends treating links as hostile and limiting web tools. <a href="#">[55]</a> |
| V2     | ?           | exec with host = gateway            | REQUIRE_CONSENT or DENY_POLICY          | Host exec is high risk; OpenClaw uses exec approvals and warns about tool blast radius. <a href="#">[56]</a>          |
| V3     | ?           | repeated identical process.poll 25x | DENY_BUDGET + transition to <REDACTED>  | Mirrors OpenClaw loop detection intent; ASD should enforce via budgets even if                                        |

| Vector | Prior phase | Proposed action       | Expected decision | Rationale                                                                                                         |
|--------|-------------|-----------------------|-------------------|-------------------------------------------------------------------------------------------------------------------|
| V4     | <REDACTED>  | consent token expired | HALT              | <p>loop detection disabled. [11]</p> <p>Gate semantics require expiry to prevent replay (nonce binding). [57]</p> |

### Unit tests

- Pure governor tests: (policy, state, proposed\_event) → deterministic response hash.
- Schema tests: validate ASDState, decide requests/responses against JSON Schema.
- Signature tests: verify nonce/signature validation and idempotency behavior.

### Integration tests

- Spin up OpenClaw Gateway + ASD plugin + ASD sidecar in a local docker-compose stack.
- Run “known tasks” and assert:
  - tool-call counts and recursion depth stay within configured budgets,
  - irreversible actions cannot occur without consent,
  - audit log is append-only and signatures verify.

## Security Controls, Deployment, Evaluation, and Governance Roadmap

### Security controls

#### Input and content handling

OpenClaw recommends treating links, attachments, and pasted instructions as hostile; and emphasizes that prompt injection can enter through untrusted content even when only trusted users can DM the bot. [28] ASD operationalizes this by requiring:

- `trust.inputTrustLevel` classification at 2,
- strict policy: untrusted content cannot directly reach irreversible actions,
- and “reader-agent pattern”: summarize untrusted content with a read-only agent, then pass summaries to tool-enabled agents (OpenClaw explicitly recommends this). [58]

#### Capability tokens and least privilege

NIST’s AC-6 “least privilege” control provides a well-established baseline: grant only the access needed to accomplish tasks. [59] In GenAI terms, OWASP categorizes “Excessive Agency” as a top LLM risk where systems allow LLMs to invoke extensions/tools beyond what is necessary. [15]

ASD enforces least privilege via: - per-phase allowed tool sets, - per-agent tool budgets, - explicit capability tokens for irreversible transitions (nonce-bound, expiring), - and default-deny for control-plane tools (gateway, cron) in untrusted contexts as OpenClaw recommends. [23]

### *Append-only signed audit logs*

OpenClaw notes that session logs live on disk and that disk access is the trust boundary—implying that logs are security-sensitive and must be protected. [28] NIST’s guidance on log management emphasizes the need for effective log management practices, while NIST SP 800-53 includes controls that require protecting audit information from unauthorized access/modification/deletion (e.g., AU-9). [60]

ASD v0.1 proposes: - An append-only JSONL log in `~/.openclaw/asd/audit/*.jsonl` - Each record contains: `prevHash`, `recordHash`, and a signature (HMAC or Ed25519). - Periodic “checkpoints” can be anchored externally (optional) for tamper evidence.

(Unspecified detail): OpenClaw does not mandate a specific tamper-evident audit format; ASD defines one as part of its own spec.

### *Anomaly detection*

In addition to OpenClaw’s optional loop detection, ASD can flag: - sudden spikes in tool-call count or wall-clock, - repeated consent requests, - risk escalation trajectories (e.g., immediate jumps from `?` to irreversible actions), - and trust-policy violations.

OpenClaw already documents that it can block/warn on tool-call loops (`tools.loopDetection`) and provides extensive security audit checks; ASD integrates these signals into governance. [61]

## Deployment recommendations

### *Sidecar/container patterns*

- **Local-only sidecar** (default): run the governor on `127.0.0.1`; configure firewall to prevent external access.
- **Same-host container**: run governor in Docker with no published ports and allow only Gateway container/network to reach it.
- Ensure OpenClaw is bound to loopback unless remote access is deliberately configured and secured (OpenClaw’s default and security posture). [62]

### *Leverage OpenClaw sandboxing*

OpenClaw can run tools inside Docker to reduce blast radius, with configurable modes, scopes, and workspace access, while noting it is not a perfect security boundary but materially limits access. [63] ASD recommends:

- Enable sandboxing for any agent that touches untrusted inputs, aligning with OpenClaw guidance. [64]

- Use workspaceAccess: "ro" where possible to prevent unreviewed writes, and keep bind mounts minimal and read-only. [65]

### *Local model considerations*

OpenClaw warns that local deployments require strong defenses and large context, and that small/quantized models can increase prompt-injection risk; it recommends using strong models for tool-enabled agents. [27] ASD therefore recommends:

- If running local models, tighten tool policy, enforce sandboxing on all sessions, and lower tool budgets until behavior is empirically validated.
- Prefer modern, instruction-hardened models for any tool-enabled agent role. [28]

## Evaluation metrics and experiments

ASD should be evaluated using both OpenClaw-native metrics and external agent benchmarks.

### *Metrics*

- **Tool-call efficiency:** tool calls per successful task; repeated-call ratio (supports OWASP “unbounded consumption” concerns). [66]
- **Recursion stability:** distribution of recursion depths; number of forced halts.
- **Irreversible safety:** rate of irreversible actions executed without valid consent (should be zero).
- **Time and cost:** wall-clock time, token usage, provider spend (OpenClaw documents multiple cost-bearing features). [67]
- **Prompt injection resilience:** success rate on known injection test cases (direct and indirect), based on OWASP prompt injection guidance and academic work. [68]

### *Experiment design*

- **A/B within OpenClaw:** baseline (tool policy + sandbox + loop detection) vs baseline + ASD governor.
- OpenClaw provides clear knobs for tool policy (profiles/groups/allow/deny) and loop detection; ASD should measure incremental improvement beyond these. [69]
- **Benchmark harness:** integrate with environments and benchmarks used for agent tool-use evaluation:
  - AgentBench (LLM-as-agent benchmark across interactive environments). [70]
  - StableToolBench/ToolBench (tool learning benchmarks). [71]
  - WebArena (realistic web environment for agents). [72]
  - SWE-bench / SWE-bench-Live for software engineering agent workflows. [73]

The goal is not to “beat” these benchmarks, but to quantify reductions in runaway behaviors and unsafe transitions under controlled workloads.

## Governance and standardization path

OpenClaw itself treats security as a configuration-validated, policy-enforced discipline with tooling such as `openclaw security audit`, and even tracks formal security models (TLA+/TLC) in a separate repository for bounded regression checking of security claims like approvals and replay resistance. [74] ASD can follow a similar path:

**ASD Spec v0.1** - Canonical glyph-to-enum mapping (<REDACTED>) - JSON Schemas for state and decision API - Deterministic reference governor ruleset (YAML or JSON policy) - Compliance test vectors (must-pass)

**v0.1 test suite** - Determinism test: stable decision hashes across runs - Schema validation tests - Policy conformance tests: irreversible actions always require a gate token - Interop tests: different UI glyph visualizations must map to same enum transitions

**Optional glyph visualizations** - OpenClaw Control UI already renders config forms and supports rich UI capabilities; ASD can add a simple “state badge” widget via message/canvas surfaces, but this is optional and should not affect governance. [75]

## Recommended next steps and implementation roadmap

### *Immediate next steps*

Adopt a phased rollout that matches OpenClaw’s “start with smallest access” guidance and uses its existing audit and sandboxing commands. [76]

gantt

```
title ASD for OpenClaw – Implementation Roadmap (v0.1)
dateFormat YYYY-MM-DD
axisFormat %b %d
```

#### section Foundation

```
Define v0.1 glyph/enums + schemas      :a1, 2026-02-20, 7d
Build reference governor (pure)        :a2, after a1, 14d
Compliance vectors + determinism tests :a3, after a2, 10d
```

#### section OpenClaw Integration

```
Proxy-tool plugin (asd_execute)        :b1, 2026-03-01, 14d
Strict tool policy profiles + docs      :b2, after b1, 7d
Local-only sidecar deployment          :b3, after b1, 7d
```

#### section Security Hardening

```
Consent-gate UX + token binding        :c1, after b3, 10d
Append-only signed audit log           :c2, after c1, 10d
Anomaly detection + alerts             :c3, after c2, 14d
```

#### section Evaluation

```
A/B testing on internal tasks          :d1, 2026-04-01, 14d
Benchmark harness integration          :d2, after d1, 21d
```

### *Practical OpenClaw configuration baseline for ASD pilots*

Start from OpenClaw’s hardened security baseline—local-only gateway, per-peer DM isolation, messaging tool profile, deny high-risk tool groups—and then add ASD tools selectively. [45] This aligns with OpenClaw’s own recommendation to tighten who can talk to the bot and minimize tool blast radius as the primary security posture. [3]

### *Final note on “real change” and responsible dissemination*

If your goal is “real change” for OpenClaw users, ASD’s most impactful contribution is not the glyphs themselves, but **a reproducible, testable standard** (schemas + deterministic rules + compliance suite) that integrates cleanly with OpenClaw’s security-first design philosophy and tool policy model. OpenClaw already encourages disciplined configuration, auditing, and minimizing blast radius; ASD can become a shared, portable governance layer that makes these practices *measurable and enforceable*. [77]

---

[1] GitHub - openclaw/openclaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way.

<https://github.com/openclaw/openclaw>

[2] [3] [6] [10] [14] [16] [17] [20] [22] [23] [24] [26] [28] [31] [42] [45] [47] [51] [55] [56] [58] [64] [74] [76] [77] Security - OpenClaw

<https://docs.openclaw.ai/gateway/security>

[4] [11] [13] [33] [37] [39] [40] [44] [61] [69] Tools - OpenClaw

<https://docs.openclaw.ai/tools>

[5] [30] Plugin hooks (agent\_end, before\_tool\_call, etc.) are never invoked · Issue #5513 · openclaw/openclaw · GitHub

<https://github.com/openclaw/openclaw/issues/5513>

[7] [60] SP 800-92, Guide to Computer Security Log Management | CSRC

[https://csrc.nist.gov/pubs/sp/800/92/final?utm\\_source=chatgpt.com](https://csrc.nist.gov/pubs/sp/800/92/final?utm_source=chatgpt.com)

[8] [62] Gateway Runbook - OpenClaw

<https://docs.openclaw.ai/gateway>

[9] [36] [41] Agent Loop - OpenClaw

<https://docs.openclaw.ai/concepts/agent-loop>

[12] [66] OWASP LLM10: 2025 Unbounded Consumption - OWASP Top 10 for LLM ...

[https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/?utm\\_source=chatgpt.com](https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/?utm_source=chatgpt.com)

[15] LLM06:2025 Excessive Agency - OWASP Gen AI Security Project

[https://genai.owasp.org/llmrisk/llm062025-excessive-agency/?utm\\_source=chatgpt.com](https://genai.owasp.org/llmrisk/llm062025-excessive-agency/?utm_source=chatgpt.com)

[18] [19] [29] Prompt injection is not SQL injection (it may be worse)

[https://www.ncsc.gov.uk/pdfs/blog-post/prompt-injection-is-not-sql-injection.pdf?utm\\_source=chatgpt.com](https://www.ncsc.gov.uk/pdfs/blog-post/prompt-injection-is-not-sql-injection.pdf?utm_source=chatgpt.com)

[21] How Microsoft defends against indirect prompt injection attacks

[https://www.microsoft.com/en-us/msrc/blog/2025/07/how-microsoft-defends-against-indirect-prompt-injection-attacks?utm\\_source=chatgpt.com](https://www.microsoft.com/en-us/msrc/blog/2025/07/how-microsoft-defends-against-indirect-prompt-injection-attacks?utm_source=chatgpt.com)

[25] [38] [53] Plugins - OpenClaw

<https://docs.openclaw.ai/tools/plugin>

[27] Local Models - OpenClaw

<https://docs.openclaw.ai/gateway/local-models>

[32] [59] AC-6 LEAST PRIVILEGE | NIST SP 800-53

[https://nist-sp-800-53-r5.bsafes.com/docs/3-1-access-control/ac-6-least-privilege/?utm\\_source=chatgpt.com](https://nist-sp-800-53-r5.bsafes.com/docs/3-1-access-control/ac-6-least-privilege/?utm_source=chatgpt.com)

[34] [43] [48] [57] Gateway Protocol - OpenClaw

<https://docs.openclaw.ai/gateway/protocol>

[35] Plugin Agent Tools - OpenClaw

<https://docs.openclaw.ai/plugins/agent-tools>

[46] [50] [52] [54] Tools Invoke API - OpenClaw

<https://docs.openclaw.ai/gateway/tools-invoke-http-api>

[49] [75] Gateway Architecture - OpenClaw

<https://docs.openclaw.ai/concepts/architecture>

[63] [65] Sandboxing - OpenClaw

<https://docs.openclaw.ai/gateway/sandboxing>

[67] API Usage and Costs - OpenClaw

[https://docs.openclaw.ai/reference/api-usage-costs?utm\\_source=chatgpt.com](https://docs.openclaw.ai/reference/api-usage-costs?utm_source=chatgpt.com)

[68] LLM Prompt Injection Prevention - OWASP Cheat Sheet Series

[https://cheatsheetseries.owasp.org/cheatsheets/LLM\\_Prompt\\_Injection\\_Prevention\\_Cheat\\_Sheet.html?utm\\_source=chatgpt.com](https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html?utm_source=chatgpt.com)

[70] [2308.03688] AgentBench: Evaluating LLMs as Agents - arXiv.org

[https://arxiv.org/abs/2308.03688?utm\\_source=chatgpt.com](https://arxiv.org/abs/2308.03688?utm_source=chatgpt.com)

[71] StableToolBench: Towards Stable Large-Scale Benchmarking on Tool ...

[https://arxiv.org/abs/2403.07714?utm\\_source=chatgpt.com](https://arxiv.org/abs/2403.07714?utm_source=chatgpt.com)

[72] WebArena: A Realistic Web Environment for Building Autonomous Agents

[https://arxiv.org/abs/2307.13854?utm\\_source=chatgpt.com](https://arxiv.org/abs/2307.13854?utm_source=chatgpt.com)

[73] SWE-bench: Can Language Models Resolve Real-World GitHub Issues?

[https://arxiv.org/abs/2310.06770?utm\\_source=chatgpt.com](https://arxiv.org/abs/2310.06770?utm_source=chatgpt.com)